Progress on the FPGA Implementation project

-Amartya T.

My Work

- I have been assigned 2 main modules. A **CORDIC algorithm** based **Square root** function, and a beamforming function.
- The **CORDIC square root** is a simple but effective hardware oriented algorithm to calculate the square root of any binary number. I have synthesized and implemented it successfully in Verilog, taking an input number and resulting in an output.
- The beamforming function implements a **Delay-and-Sum (DAS)** architecture that computes time-of-flight for 128 transducer elements and essentially “focuses” the echoes off the microbubble by delaying each signal to cause constructive interference. It uses the **CORDIC square root** to derive precise RF memory addresses for real-time pixel reconstruction across a given image grid. It currently calculates 51 addresses for a single pixel at a time, with a delay of roughly **50 clock cycles per pixel**, which can be optimized quite a bit.
  The **RF address** is basically a digital pointer that tells the hardware “where” to grab the delay data from.

Weekly Progress Timeline

- **Weeks 1–4a: CORDIC Square Root**
- The goal here is a bit-accurate, and an efficient square-root calculator that takes an input number and returns its squareroot.
- **Weeks 1–2:** Learn the CORDIC algorithm, study the MATLAB code and understand how it works. Make a rough vivado implementation.
- **Weeks 3–4:** Verify RTL, test rigorously, optimize algorithm and try to synthesize & implement.

- **Weeks 5–6: Delay Calculator and CORDIC Implementation**
- Using CORDIC module try to find the propagation delay of the signal to perform beamforming correctly.
- **Week 5:** Map the input data to the CORDIC input (x2+y2) to calculate distance.
- **Week 6:** Break the Beamforming function into a 3 module pipeline, work from the bottom up.

- **Weeks 7–8: Beamformer Integration**
- The final "Delay and Sum" (DAS) stage.
- **Week 7:** Build the Delay Calculator and test it against the MATLAB module. Make a rough skeleton of the beamformer operation.
- **Week 8:** Synthesize and implement the Delay Calculator alone and then try to use beamformer to call M instances in parallel to calculate all delays for a pixel.

Testbench values of CORDIC sqrt function:

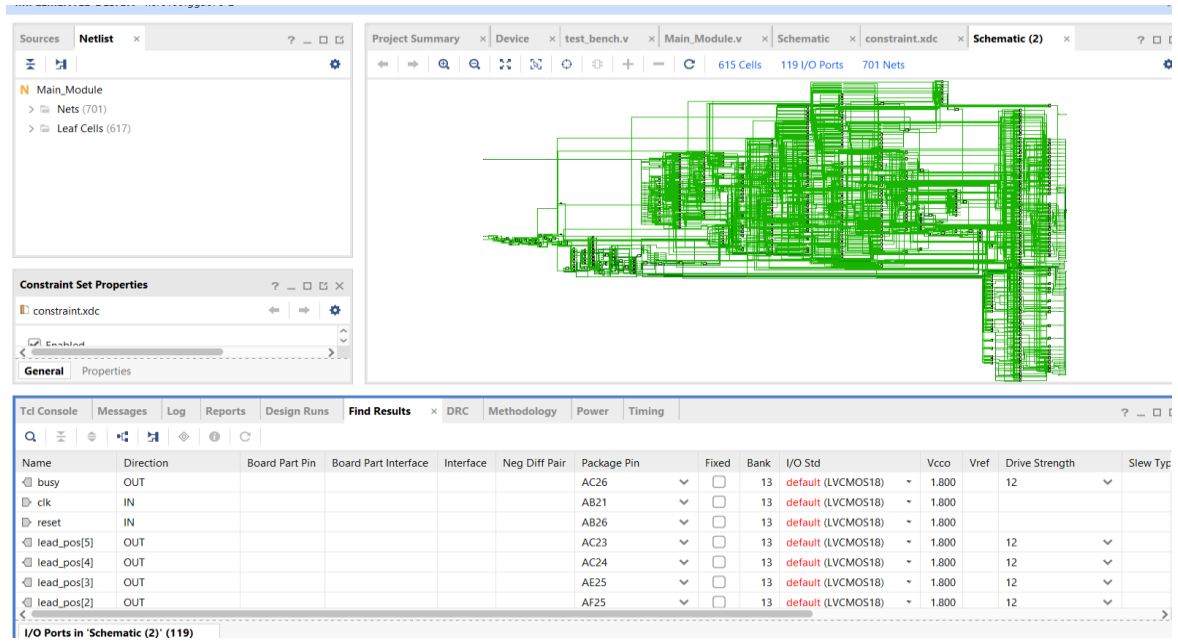
- Keeping in mind that the values of Verilog are using a number system **Q 6.26** (6 integer bits, 26 fractional bits, and has an input range of upto **63.99**) It has reasonably good accuracy, but the method used has its limitations in Verilog because the number of iterations are capped relative to the widths of the numbers. Accuracy can be increased as I have parametrized all the inputs, so the code should be scalable for larger number systems giving closer results. **RMS error** of the table below is : **0.0587%**.

Input (n)	Expected	Actual	Abs Error	%% Error
397050	630.119	630.000	-0.119	-0.019%
1795748	1340.055	1340.000	-0.055	-0.004%
476578	690.346	690.000	-0.346	-0.050%
735972	857.888	857.000	-0.888	-0.104%
763410	873.733	873.000	-0.733	-0.084%

Input (n)	Expected	Actual	Abs Error	%% Error
396921	630.017	630.000	-0.017	-0.003%
909950	953.913	953.000	-0.913	-0.096%
1257865	1121.546	1121.000	-0.546	-0.049%
1400788	1183.549	1183.000	-0.549	-0.046%
980209	990.055	990.000	-0.055	-0.006%

Synthesis & Implementation

- Synthesis was done successfully, and implementation was also successfully run.
- There are some warnings related to clock and timing, but those can only be tweaked once I have the board that I want to implement it on, ready to go.
- The implemented design uses 118 IO ports, has 352 LUTs and 55 flip flops.

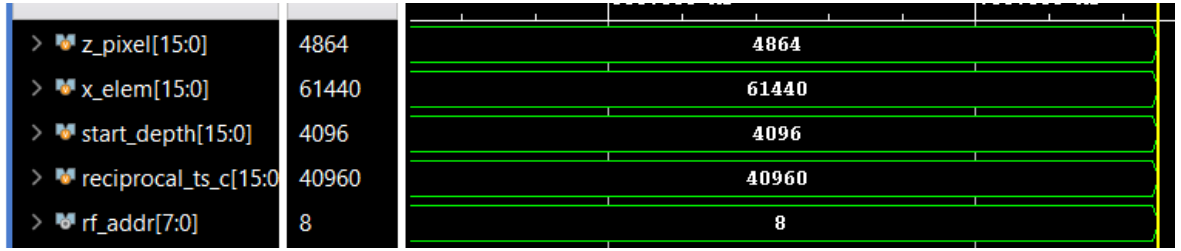


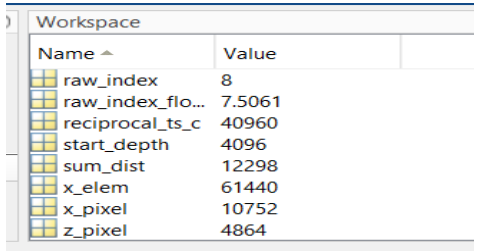
Beamforming Function

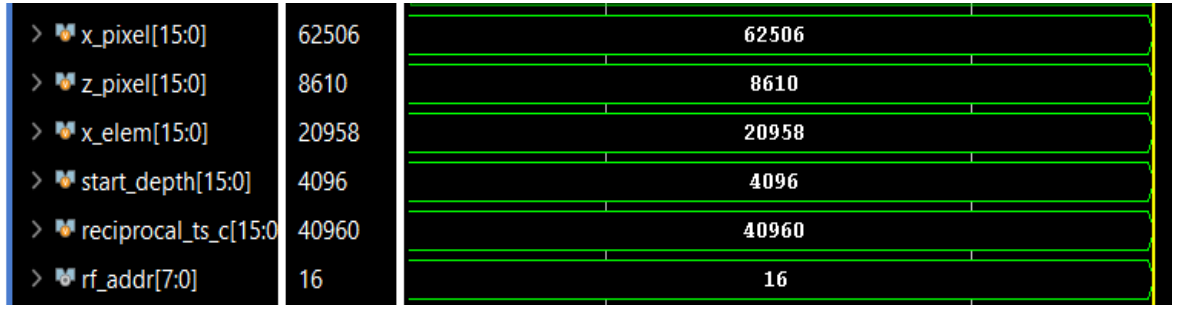
- The beam forming function is not as straightforward as the CORDIC sqrt. To start, I have made a **Delay calculator** mimicking the matlab code that calculates the RF address value for a single channel for a given pixel. I then run M of these instances in parallel(parametrized so can be changed), to calculate all the M address values for a single pixel within 50-55 clock cycles(Where M is the size of the aperture window). Lastly, there is a **Top level controller** so that I can pass each of the pixel values to the beamforming function sequentially, so that it may calculate the whole image grid. The controller also contains a **co-ordinate generator**, that reads each pixel values from the **ROM**.

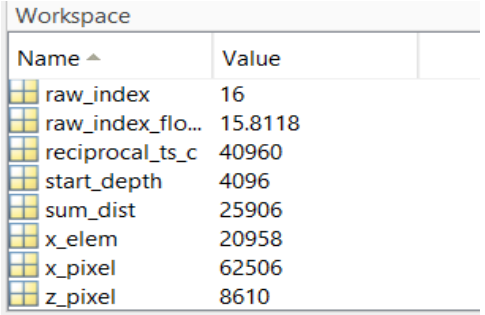
Delay Calculator Test Bench

- The Delay Calculator produces the accurate RF index for each of the testing values. I made a simple matlab function that simulates the delay calculation based on the original code to cross-verify my Verilog test-bench values. I truncated the raw_index_float value, which is sliced from the integer part of my K_n_long, which is a signal that contains the calculations from the given formula in the MATLAB code.



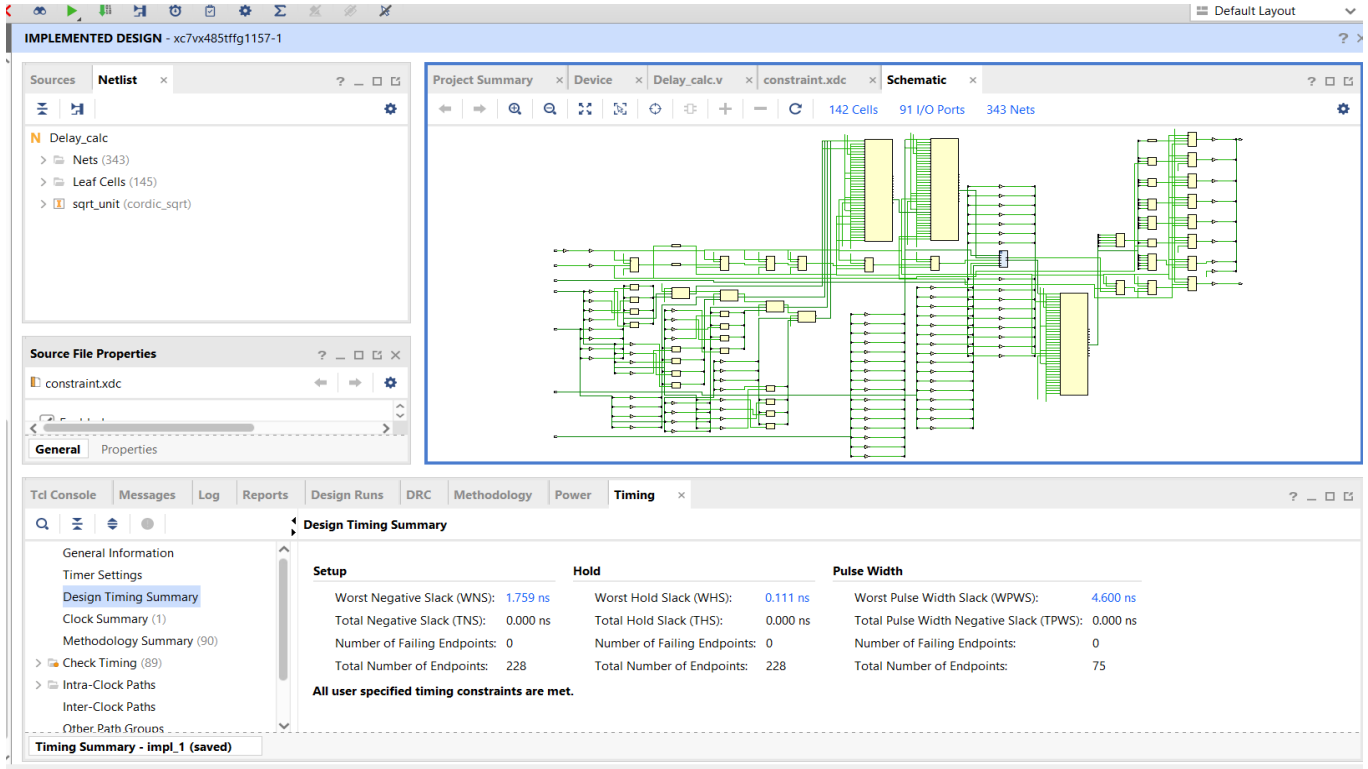






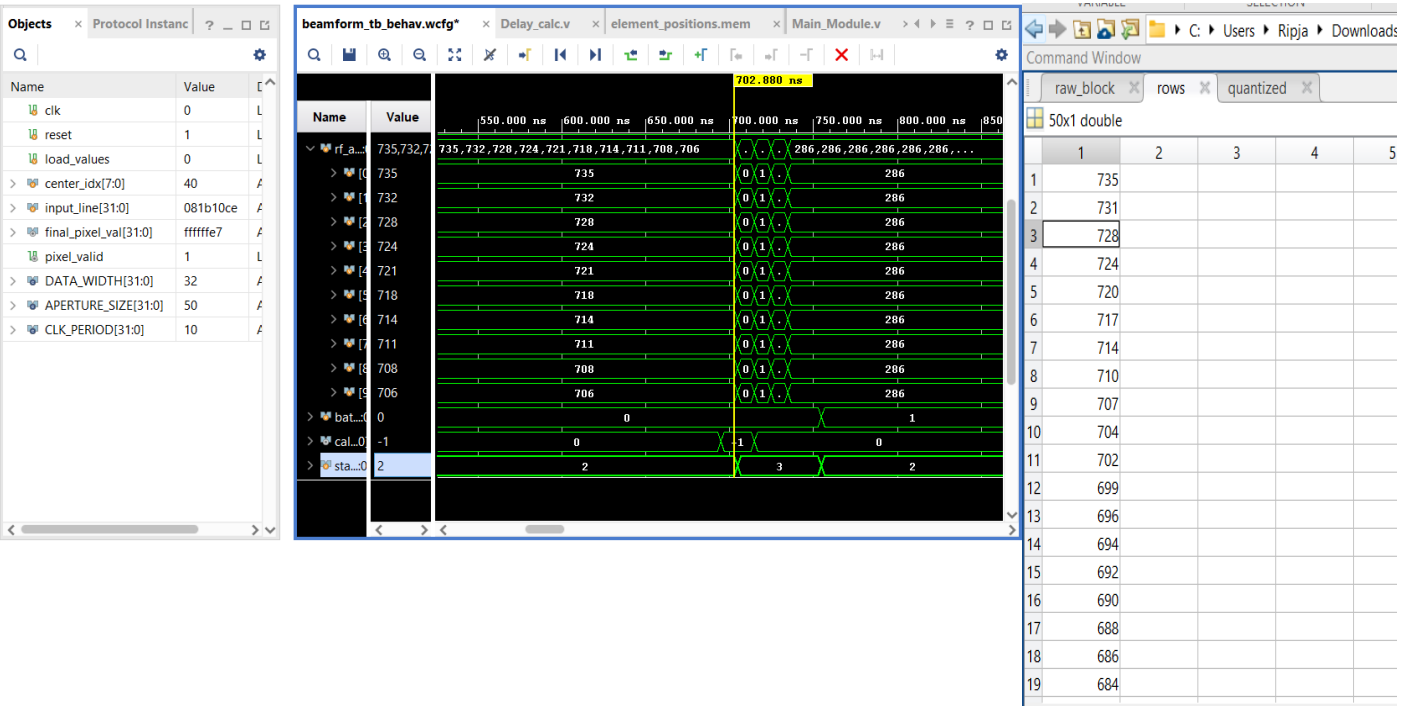
Delay Calculator Synthesis & Implementation

- The Synthesis and implementation both of the Delay Calculator were successful. Here is the implemented design:
- The design uses uses 312 LUTs and 71 FFs with 91 I/O ports.

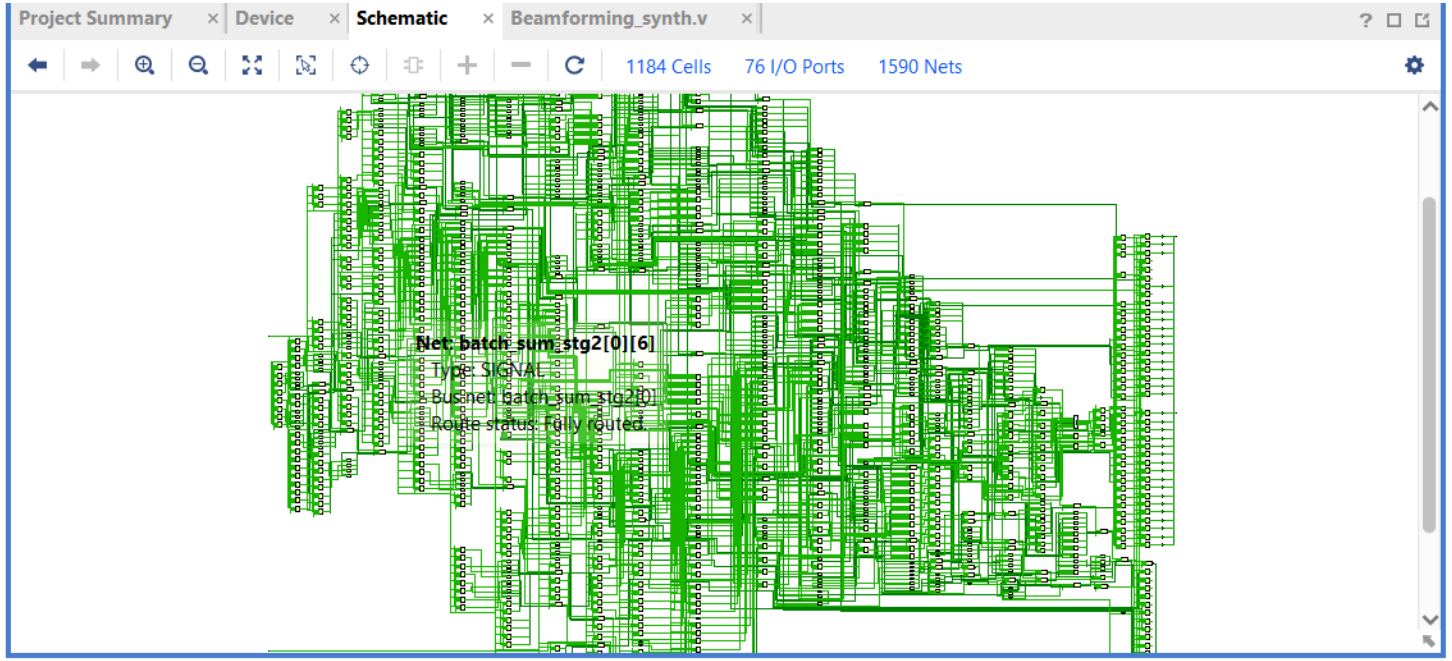


Beamforming Testing and Implementation

- The Beamforming function produces 50 RF addresses and goes to retrieve sampled data from each of these addresses. This sampled data is then used to calculate the final pixel brightness of each pixel. It instantiates a Delay calculator to find the RF addresses, using the square root for distance calculations. The simulation results are shown below. Implementation was also successful, shown on the next slide.



Beamforming Synthesis



Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP
constrs_1	synth_design Complete!												505	565	0	0	3
constrs_1	**route_design Complete!**	**0.596**	**0.000**	**0.160**	**0.000**		**0.000**	**0.100**	**0**	**282 Warn**			**435**	**565**	**0**	**0**	**3**
constrs_1	Not started																

Future Plan

- Integration with other modules to fit into the rest of the pipeline.
- Optimization of CORDIC square root, reducing no. of clock cycles used, and optimization of the beamforming function.
- Implementation of BRAM along with the modules to allow the data to be directly loaded and processed on the FPGA alone without the need for external devices. Or alternatively, implementing UART to connect to a host, performing calculations and returning values directly back to the host.

Thank You!